

Lab Task: intArrayList - Array-Based List Implementation

You are given a skeleton C++ file (intArrayList_skeleton.cpp) with the class declaration, some functions already implemented, and the rest left for you to write. This document explains what every function must do, in the order they are used in main(). Read each explanation carefully before writing code, and check your output against the example given.

Rules: Do not change any function signature. Do not change main(). Do not use std::vector or any STL container -- use the raw pointer 'list' and plain arrays only. Every error case mentioned below must print a message to the screen and must not crash the program.

1. Data Members

The class stores the list using three protected data members:

```
int* list;      // pointer to a dynamically allocated array of ints
int length;     // number of elements currently stored in the list
int maxSize;    // maximum number of elements the array can hold
```

Every function below works by reading or changing these three members.

2. Constructor -- intArrayList(int size = 100)

Called when an object is created, e.g. intArrayList myList(10) creates a list with room for 10 integers.

What to do:

1. Set maxSize = size.
2. Set length = 0 (the list starts empty).
3. Dynamically allocate the array: list = new int[size];

Example:

intArrayList myList(10); creates an object where maxSize is 10, length is 0, and list points to a newly allocated array that can hold 10 integers (currently empty).

3. isEmpty() -- already implemented

Returns true if length == 0, false otherwise. This one is done for you; study it to understand the pattern used by the other functions.

4. isFull()

Returns true if the list has no more room, false otherwise.

What to do:

Return the result of comparing length to maxSize. The list is full when length has reached maxSize (no more slots left).

Example:

If maxSize is 10 and length is 10, isFull() returns true. If length is 7, isFull() returns false.

5. `maxListSize()`

Returns the maximum capacity of the list.

What to do:

Simply return `maxSize`. (This is exactly like `listSize()`, which returns `length` -- already implemented for you above it.)

6. `insertEnd(int insertItem)`

Called three times in `main()`: `myList.insertEnd(10)`; `myList.insertEnd(20)`; `myList.insertEnd(30)`; This adds an item to the end of the list, after whatever is already there.

What to do:

1. If the list is full (`length == maxSize`), print an error message such as "Cannot insert, list is full" and do nothing else.
2. Otherwise, place `insertItem` at position `list[length]`, then increase `length` by 1.

Example:

Start: list is empty, `length = 0`.

`insertEnd(10)` -> list = [10], `length` becomes 1.

`insertEnd(20)` -> list = [10, 20], `length` becomes 2.

`insertEnd(30)` -> list = [10, 20, 30], `length` becomes 3.

This is why the expected output after these three calls is: 10 20 30

7. `print()` -- already implemented

Loops from index 0 to `length - 1` and prints each element followed by a space. This is already written for you and is used throughout `main()` to show the list contents after every change.

8. `insertAt(int location, int insertItem)`

Called as: `myList.insertAt(1, 99)`; Unlike `insertEnd`, this inserts an item at a specific position, and everything from that position onward must shift one slot to the right to make room.

What to do:

1. If `location` is out of range (`location < 0` or `location > length`) OR the list is full, print an error message and do nothing else.
2. Otherwise, starting from the last used index and moving down to `location`, copy each element one position to the right: `list[i] = list[i - 1]`, for `i` going from `length` down to `location + 1`.
3. Place `insertItem` at `list[location]`.
4. Increase `length` by 1.

Example:

Before: list = [10, 20, 30], `length = 3`.

Call: `insertAt(1, 99)` -- insert 99 at location 1.

Step 1: shift `list[2]` (which is 30) into `list[3]`: list = [10, 20, 30, 30]

Step 2: shift `list[1]` (which is 20) into `list[2]`: list = [10, 20, 20, 30]

Step 3: place 99 at list[1]: list = [10, 99, 20, 30]

Step 4: length becomes 4.

Final result: 10 99 20 30 -- notice 99 is now at location 1, and 20 and 30 moved one step to the right to make room.

9. seqSearch(int item) const

Called twice: myList.seqSearch(30) and myList.seqSearch(500). This searches the list from the beginning for a given value.

What to do:

1. Loop through the list from index 0 to length - 1.
2. If list[i] equals item, return i immediately.
3. If the loop finishes without finding the item, return -1.

Example:

If list = [10, 99, 20, 30], seqSearch(30) returns 3 (30 is found at index 3).

seqSearch(500) returns -1, because 500 does not exist anywhere in the list.

10. removeAt(int location)

Called as: myList.removeAt(0);. This removes the item at a specific position and shifts every element after it one slot to the left, so there is no empty gap left behind.

What to do:

1. If location is out of range (location < 0 or location >= length), print an error message and do nothing else.
2. Otherwise, starting from location and moving up to the second-last used index, copy each element one position to the left: list[i] = list[i + 1], for i going from location to length - 2.
3. Decrease length by 1.

Example:

Before: list = [10, 99, 20, 30], length = 4.

Call: removeAt(0) -- remove the element at location 0 (which is 10).

Step: shift list[1] into list[0], list[2] into list[1], list[3] into list[2]:

list becomes [99, 20, 30, 30] (the last 30 is left over from the old array, but it no longer counts, because length is now 3).

Final visible result (only first 3 elements matter now): 99 20 30

11. retrieveAt(int location, int& retItem) const

Called as: myList.retrieveAt(0, val);. This copies the value at a given position into a variable supplied by the caller, without removing it from the list.

What to do:

1. If location is out of range ($\text{location} < 0$ or $\text{location} \geq \text{length}$), print an error message and do nothing else.
2. Otherwise, set $\text{retItem} = \text{list}[\text{location}]$.

Example:

If $\text{list} = [99, 20, 30]$ and we call $\text{retrieveAt}(0, \text{val})$, then val becomes 99. The list itself is unchanged.

12. replaceAt(int location, int replItem)

Called as: $\text{myList.replaceAt}(0, 777)$; This overwrites the value at a given position with a new value.

What to do:

1. If location is out of range ($\text{location} < 0$ or $\text{location} \geq \text{length}$), print an error message and do nothing else.
2. Otherwise, set $\text{list}[\text{location}] = \text{replItem}$.

Example:

If $\text{list} = [99, 20, 30]$ and we call $\text{replaceAt}(0, 777)$, the list becomes $[777, 20, 30]$. Only the value at location 0 changes; length does not change.

13. insert(int insertItem)

Called twice: $\text{myList.insert}(777)$; then $\text{myList.insert}(555)$; This is different from insertEnd -- it first checks whether the item is already in the list, and only inserts it at the end if it is not already present.

What to do:

1. Use seqSearch to check if insertItem already exists in the list.
2. If it already exists, print a message such as "Item already in list" and do nothing else.
3. If the list is full, print an error message and do nothing else.
4. Otherwise, insert it at the end exactly like insertEnd does (place at $\text{list}[\text{length}]$, then $\text{length}++$).

Example:

Suppose $\text{list} = [777, 20, 30]$ before these calls (777 is already present from the previous step).
 $\text{insert}(777)$ -- 777 is already in the list, so nothing is inserted; a message is printed instead.
 $\text{insert}(555)$ -- 555 is not in the list, so it is inserted at the end: $\text{list} = [777, 20, 30, 555]$.

14. remove(int removeItem)

Called as: $\text{myList.remove}(20)$; This searches for a specific value and removes it if found.

What to do:

1. Use seqSearch to find the index of removeItem .
2. If it is not found (seqSearch returns -1), print a message such as "Item not found" and do nothing else.
3. If it is found at some index, call removeAt on that index to actually remove it.

Example:

If list = [777, 20, 30, 555] and we call remove(20), seqSearch finds 20 at index 1, so removeAt(1) is called internally. The list becomes [777, 30, 555].

15. Copy Constructor -- intArrayList(const intArrayList& otherList)

Called as: intArrayList copyList = myList;. This creates a brand new object that is an independent copy of an existing list -- not just a copy of the pointer, but a copy of the actual array contents.

Why this matters:

If you only copied the pointer (list = otherList.list), both objects would share the exact same array in memory. Changing one would change the other, and when one object is destroyed, its destructor would free memory the other object still needs. This is called a shallow copy and it is wrong. You must do a deep copy instead.

What to do:

1. Set maxSize = otherList.maxSize and length = otherList.length.
2. Allocate a brand new array: list = new int[maxSize];
3. Copy each element one by one from otherList.list into your new list, for indices 0 to length - 1.

Example:

If myList = [777, 30, 555] with maxSize 10, then intArrayList copyList = myList; must produce a copyList with its own separate array also containing [777, 30, 555]. If you later change myList, copyList must stay [777, 30, 555] because it is a completely separate copy.

16. operator= -- const intArrayList& operator=(const intArrayList& otherList)

Called as: assignedList = myList;, where assignedList already exists (it was created earlier with intArrayList assignedList(5);). This is different from the copy constructor because the object on the left already has its own array that must be cleaned up first.

What to do:

1. Check for self-assignment: if (this == &otherList) return *this; (this handles the case someone writes myList = myList;).
2. Free the array you currently have: delete[] list;
3. Set maxSize = otherList.maxSize and length = otherList.length.
4. Allocate a new array: list = new int[maxSize];
5. Copy every element from otherList.list into your new list, for indices 0 to length - 1.
6. Return *this;

Example:

Before: assignedList has its own array of size 5, empty (length 0).

myList = [777, 30, 555] with maxSize 10.

After assignedList = myList;, assignedList's old array (size 5) is deleted, a new array of size 10 is allocated, and assignedList now independently holds [777, 30, 555]. Changing myList afterward must not affect assignedList.

17. clearList()

Called as: `myList.clearList();`. This empties the list without deleting the underlying array.

What to do:

Simply set `length = 0`. You do NOT need to delete the array here -- the array stays allocated (with `maxSize` capacity); it is just treated as having no valid elements anymore. Old values may still physically sit in memory, but since `length` is 0, no function will read or print them.

Example:

If `list = [777, 30, 555]` with `length 3`, after `clearList()`, `length` becomes 0. `isEmpty()` will now return true, and `print()` will output nothing.

18. isItemAtEqual(int location, int item) const

This function is not called directly in `main()`, but you must still implement it, and it may be tested separately.

What to do:

Return true if `list[location]` equals `item`, false otherwise. Assume `location` is always valid for this function (no need to check bounds).

Example:

If `list = [10, 99, 20, 30]`, `isItemAtEqual(1, 99)` returns true, and `isItemAtEqual(1, 50)` returns false.

19. Destructor -- ~intArrayList()

Not called explicitly anywhere in `main()`, but it runs automatically for every object when it goes out of scope at the end of `main()` (for `myList`, `copyList`, and `assignedList`).

What to do:

Free the dynamically allocated array: `delete[] list;`. This prevents memory leaks.

Submission Reminder

Implement every function described above inside `intArrayList_skeleton.cpp`, in the space marked with `/// write ur code here`. Do not modify `main()` or any function signature. Compile and run your file, and compare your output line by line against the expected results described in the examples above before submitting.